

# WEBRUM-03: Core Web Vitals

---

**Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 3 of 10 | **Created:** March 2026 | **Last Updated:** 08/03/2026

## Overview

---

Core Web Vitals (CWV) are Google's standardized metrics for measuring real-world user experience on the web. They focus on three pillars: loading performance, interactivity, and visual stability. Dynatrace captures these metrics via the RUM JavaScript agent using the browser's PerformanceObserver API, making them available for DQL analysis in Grail.

---

## Table of Contents

---

1. [Understanding Core Web Vitals](#)
  2. [Largest Contentful Paint \(LCP\)](#)
  3. [Interaction to Next Paint \(INP\)](#)
  4. [Cumulative Layout Shift \(CLS\)](#)
  5. [CWV by Page Group](#)
  6. [CWV Trends Over Time](#)
  7. [CWV Scoring Dashboard](#)
  8. [Summary and Next Steps](#)
-

# Prerequisites

| Requirement           | Details                                                                         |
|-----------------------|---------------------------------------------------------------------------------|
| Dynatrace Environment | SaaS with Grail enabled                                                         |
| RUM Enabled           | Web applications with Core Web Vitals capture enabled                           |
| Browser Support       | CWV are Chromium-based only (Chrome, Edge); Safari/Firefox have partial support |
| Permissions           | <code>storage:events:read</code> , <code>storage:metrics:read</code>            |
| Previous Notebook     | WEBRUM-01: RUM Fundamentals                                                     |

## 1. Understanding Core Web Vitals

Google defines three Core Web Vitals that reflect real user experience:

| Metric | Full Name                 | Measures                     | Good         | Needs Improvement | Poor      |
|--------|---------------------------|------------------------------|--------------|-------------------|-----------|
| LCP    | Largest Contentful Paint  | Loading performance          | $\leq 2.5s$  | 2.5s – 4.0s       | $> 4.0s$  |
| INP    | Interaction to Next Paint | Interactivity responsiveness | $\leq 200ms$ | 200ms – 500ms     | $> 500ms$ |
| CLS    | Cumulative Layout Shift   | Visual stability             | $\leq 0.1$   | 0.1 – 0.25        | $> 0.25$  |

**Note:** INP replaced FID (First Input Delay) as a Core Web Vital in March 2024. INP measures the latency of *all* interactions during a session, not just the first one.

### How Dynatrace Captures CWV

The RUM JavaScript agent uses the browser's `PerformanceObserver` API to capture:

- **LCP** — Observed via `largest-contentful-paint` entry type

- **INP** — Calculated from `event` entry types (click, keypress, pointerdown)
- **CLS** — Accumulated from `layout-shift` entries (excluding user-initiated shifts)

These values are reported as part of the user action data and are available as metrics in Grail.

## 2. Largest Contentful Paint (LCP)

---

LCP measures the time from when the user initiates navigation to when the largest content element (image, text block, video) is rendered in the viewport. It answers: **"How quickly does the main content appear?"**

### Common LCP Elements

| Element Type                      | Example                   |
|-----------------------------------|---------------------------|
| <code>&lt;img&gt;</code>          | Hero image, product photo |
| <code>&lt;video&gt;</code> poster | Video thumbnail           |
| Block-level text                  | Heading, paragraph        |
| CSS <code>background-image</code> | Banner background         |

### Factors That Impact LCP

- Server response time (TTFB)
- Render-blocking resources (CSS, JS)
- Resource load time (images, fonts)
- Client-side rendering delays

```
// Average LCP across all applications in the last 24 hours
fetch user.events, from:-24h
| filter action.type == "Load"
| filter isNotNull(web_vitals.largest_contentful_paint)
| summarize avg_lcp = avg(web_vitals.largest_contentful_paint),
  p75_lcp = percentile(web_vitals.largest_contentful_paint, 75),
  p95_lcp = percentile(web_vitals.largest_contentful_paint, 95),
  sample_size = count(),
  by:{app.name}
| sort avg_lcp desc
```

```
// LCP distribution – classify into Good / Needs Improvement / Poor
fetch user.events, from:-24h
| filter action.type == "Load"
| filter isNotNull(web_vitals.largest_contentful_paint)
| fieldsAdd lcp_ms = web_vitals.largest_contentful_paint / 1ms
| fieldsAdd lcp_category = if(lcp_ms <= 2500, "Good",
  else: if(lcp_ms <= 4000, "Needs Improvement",
  else: "Poor"))
| summarize action_count = count(), by:{lcp_category}
| fieldsAdd total = sum(action_count)
| fieldsAdd percentage = round(toDouble(action_count) / toDouble(total) *
100.0, decimals: 1)
```

### 3. Interaction to Next Paint (INP)

---

INP measures the time from when a user interacts (click, tap, keypress) to when the browser renders the next frame reflecting that interaction. Unlike FID (which only measured the *first* interaction), INP considers *all* interactions and reports the worst one (approximated by the 98th percentile).

#### What INP Captures

1. **Input delay** — Time from user interaction to event handler execution
2. **Processing time** — Time to execute event handlers
3. **Presentation delay** — Time from handler completion to next paint

## Common INP Bottlenecks

| Bottleneck                | Symptom                 | Fix                            |
|---------------------------|-------------------------|--------------------------------|
| Long tasks on main thread | High input delay        | Break up long JavaScript tasks |
| Expensive event handlers  | High processing time    | Debounce, use web workers      |
| Forced layout/reflow      | High presentation delay | Batch DOM reads/writes         |

```
// INP percentile analysis by application
fetch user.events, from:-24h
| filter isNotNull(web_vitals.interaction_to_next_paint)
| fieldsAdd inp_ms = web_vitals.interaction_to_next_paint / 1ms
| summarize avg_inp = avg(inp_ms),
  p75_inp = percentile(inp_ms, 75),
  p95_inp = percentile(inp_ms, 95),
  sample_size = count(),
  by:{app.name}
| sort p75_inp desc
```

```
// INP by page – identify the slowest-responding pages
fetch user.events, from:-24h
| filter isNotNull(web_vitals.interaction_to_next_paint)
| fieldsAdd inp_ms = web_vitals.interaction_to_next_paint / 1ms
| summarize avg_inp = avg(inp_ms),
  p75_inp = percentile(inp_ms, 75),
  action_count = count(),
  by:{action.name}
| filter action_count > 10
| sort p75_inp desc
| limit 10
```

## 4. Cumulative Layout Shift (CLS)

CLS measures unexpected layout shifts during the entire lifespan of a page. A layout shift occurs when a visible element changes position from one rendered frame to the next without user interaction.

## Common Causes of CLS

| Cause                        | Example                                          | Fix                                                       |
|------------------------------|--------------------------------------------------|-----------------------------------------------------------|
| Images without dimensions    | Image loads and pushes content down              | Set <code>width</code> and <code>height</code> attributes |
| Dynamically injected content | Cookie banner pushes page down                   | Reserve space with CSS                                    |
| Web fonts                    | FOUT (flash of unstyled text) causes text reflow | Use <code>font-display: swap</code> with fallback metrics |
| Ads/embeds                   | Third-party content loads late                   | Use <code>&lt;iframe&gt;</code> with fixed dimensions     |

```
// CLS distribution – classify into Good / Needs Improvement / Poor
fetch user.events, from:-24h
| filter action.type == "Load"
| filter isNotNull(web_vitals.cumulative_layout_shift)
| fieldsAdd cls_category = if(web_vitals.cumulative_layout_shift <= 0.1,
"Good",
  else: if(web_vitals.cumulative_layout_shift <= 0.25, "Needs
Improvement",
    else: "Poor"))
| summarize action_count = count(), by:{cls_category}
```

```
// Worst CLS pages – pages with the most layout shifting
fetch user.events, from:-24h
| filter action.type == "Load"
| filter isNotNull(web_vitals.cumulative_layout_shift)
| summarize avg_cls = avg(web_vitals.cumulative_layout_shift),
  p75_cls = percentile(web_vitals.cumulative_layout_shift, 75),
  action_count = count(),
  by:{action.name}
| filter action_count > 10
| sort p75_cls desc
| limit 10
```

## 5. CWV by Page Group

Aggregate Core Web Vitals by page to identify which pages need optimization:

```
// All three CWV metrics by page – comprehensive page health view
fetch user.events, from:-24h
| filter action.type == "Load"
| summarize
    p75_lcp_ms = percentile(web_vitals.largest_contentful_paint / 1ms, 75),
    p75_cls = percentile(web_vitals.cumulative_layout_shift, 75),
    p75_inp_ms = percentile(web_vitals.interaction_to_next_paint / 1ms, 75),
    page_views = count(),
    by:{action.name}
| filter page_views > 20
| fieldsAdd lcp_status = if(p75_lcp_ms <= 2500, "Good", else: if(p75_lcp_ms
    <= 4000, "NI", else: "Poor")),
    cls_status = if(p75_cls <= 0.1, "Good", else: if(p75_cls <= 0.25,
    "NI", else: "Poor")),
    inp_status = if(p75_inp_ms <= 200, "Good", else: if(p75_inp_ms <=
    500, "NI", else: "Poor"))
| sort page_views desc
| limit 15
```

## 6. CWV Trends Over Time

---

Tracking CWV over time helps identify regressions after deployments, seasonal patterns, and the impact of optimizations.

```
// LCP trend over the last 7 days – hourly p75
fetch user.events, from:-7d
| filter action.type == "Load"
| filter isNotNull(web_vitals.largest_contentful_paint)
| fieldsAdd lcp_ms = web_vitals.largest_contentful_paint / 1ms
| makeTimeseries p75_lcp = percentile(lcp_ms, 75), interval:1h
```

```
// CLS trend over the last 7 days – daily p75
fetch user.events, from:-7d
| filter action.type == "Load"
| filter isNotNull(web_vitals.cumulative_layout_shift)
| makeTimeseries p75_cls = percentile(web_vitals.cumulative_layout_shift, 75),
interval:1d
```

## 7. CWV Scoring Dashboard

---

Create a single-query CWV scorecard showing the percentage of page loads in each category:

```
// CWV scorecard — percentage of Good / NI / Poor for each metric
fetch user.events, from:-24h
| filter action.type == "Load"
| fieldsAdd lcp_ms = web_vitals.largest_contentful_paint / 1ms,
  inp_ms = web_vitals.interaction_to_next_paint / 1ms
| summarize total = count(),
  lcp_good = countIf(lcp_ms <= 2500),
  lcp_poor = countIf(lcp_ms > 4000),
  cls_good = countIf(web_vitals.cumulative_layout_shift <= 0.1),
  cls_poor = countIf(web_vitals.cumulative_layout_shift > 0.25),
  inp_good = countIf(inp_ms <= 200),
  inp_poor = countIf(inp_ms > 500)
| fieldsAdd lcp_good_pct = round(toDouble(lcp_good) / toDouble(total) * 100.0,
  decimals: 1),
  lcp_poor_pct = round(toDouble(lcp_poor) / toDouble(total) * 100.0,
  decimals: 1),
  cls_good_pct = round(toDouble(cls_good) / toDouble(total) * 100.0,
  decimals: 1),
  cls_poor_pct = round(toDouble(cls_poor) / toDouble(total) * 100.0,
  decimals: 1),
  inp_good_pct = round(toDouble(inp_good) / toDouble(total) * 100.0,
  decimals: 1),
  inp_poor_pct = round(toDouble(inp_poor) / toDouble(total) * 100.0,
  decimals: 1)
```

## 8. Summary and Next Steps

---

In this notebook, we covered:

- **Core Web Vitals overview** — LCP, INP, and CLS with Google's thresholds
- **LCP analysis** — Loading performance measurement and distribution
- **INP analysis** — Interactivity measurement replacing FID
- **CLS analysis** — Visual stability scoring and worst pages
- **Page-level breakdown** — CWV per page group for targeted optimization
- **Trend tracking** — CWV over time for regression detection



- **Scoring dashboard** — Unified CWV scorecard query

## Next Steps

- **WEBRUM-04: Session Analysis** — User journey mapping and conversion tracking
- **WEBRUM-06: Performance Analysis** — Deeper performance waterfall analysis beyond CWV

## References

- [Google Core Web Vitals](#)
- [Experience Vitals \(DT docs\)](#)
- [INP Documentation](#)

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*